

Luke Merrett

Git

Different Merge Types in Git

Demystifying the difference between "Merge", "Fast Forward Merge", "Squash and Merge" and "Rebase and Merge" on Git



Luke Merrett

02 Aug 2021 • 5 min read



I was recently asked what the difference was between the 4 merging options presented to you on GitHub when finishing a PR, namely:

- Merge
- Fast Forward Merge
- Squash and Merge
- Rebase and Merge

They can be confusing for those new to Git and it's often tricky to work out which the "correct" option is at any given time. It doesn't help that the names don't give too much insight without prior knowledge of git terminology.

Firstly it's it's important to note that all of these have their own advantages and disadvantages, it's more about what best suits the team and their current needs rather than any absolute best option.

Below is a breakdown of what each type does, along with diagrams to help explain the process of each for clarity.

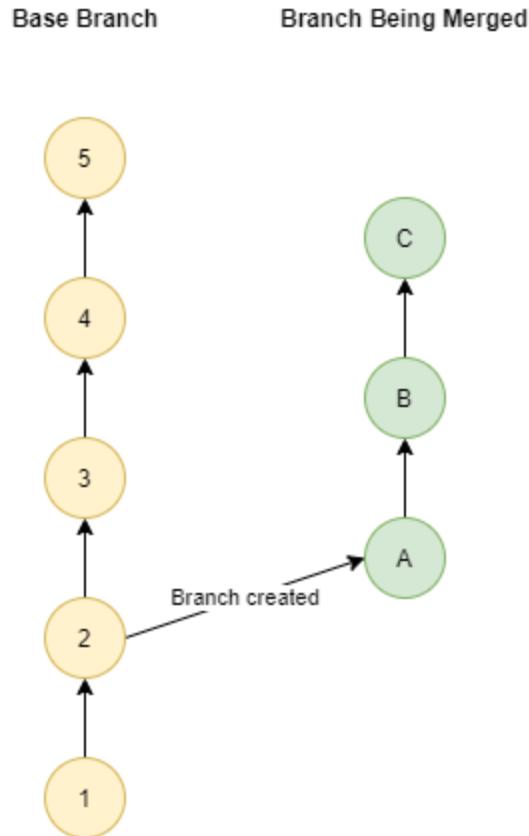
Example We'll Use

Here's what we'll use as our sample Git repository.

We have 2 branches, the base branch where the merge is going into (e.g: main, or release) and the branch being merged.

The branch being merged was created at commit **2**, so contains commits **1,2,A,B,C**

The base branch has had changes made since the new branch was made:
3,4,5.

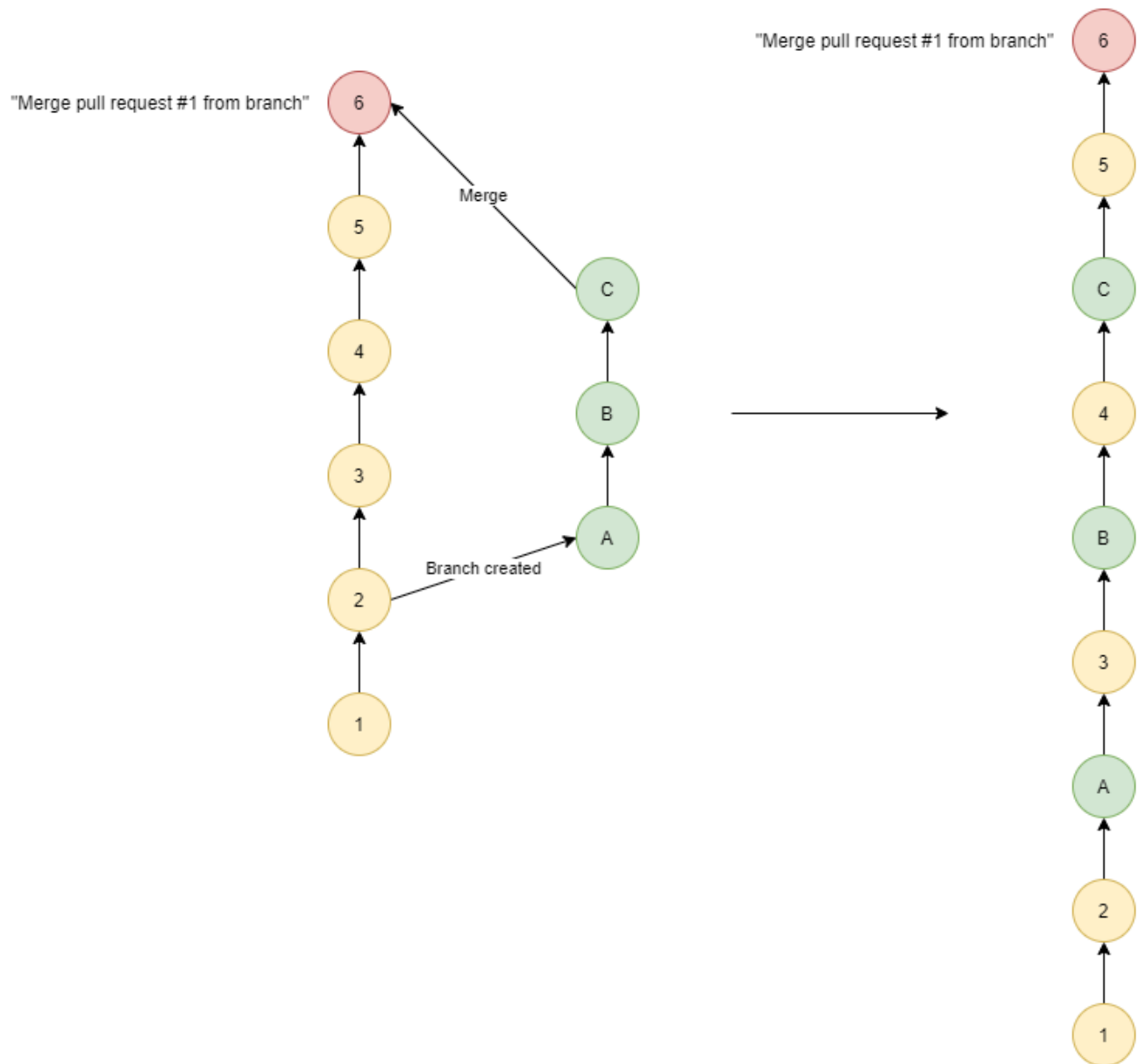


Merge Types

Merge

A standard merge will take each commit in the branch being merged and add them to the history of the base branch based on the timestamp of when they were created.

It will also create a merge commit, a special type of “empty” commit that indicates when the merge occurred



Advantages:

- Most descriptive and verbose history, tells us exactly when things happened, helps give the best context about code changes
- Allows us to see a graph of when branches were made using `git log --oneline --graph` which can help understanding why changes were made and when

- Allows us to see each commit that made up the eventual merged changes, no loss of granularity

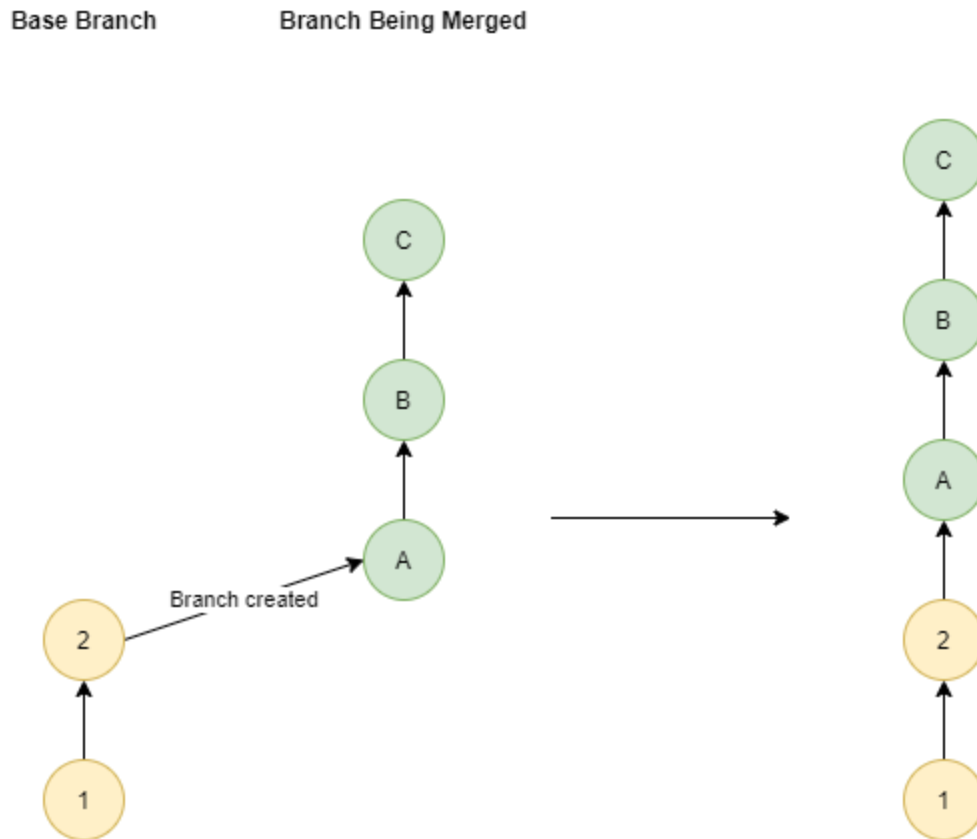
Disadvantages:

- Merge commits are often seen as messy as they are empty and only really there for historical reasons. Can be especially confusing if you are trying to revert a set of changes.
- Can end up having a complex graph of previous branches that's more difficult to read

Fast Forward Merge

If we change our example so **no new commits** were made to the base branch since our branch was created, Git can do something called a “Fast Forward Merge”. This is the same as a Merge but **does not** create a merge commit.

This is as if you made the commits directly on the base branch. The idea is because no changes were made to the base branch there's no need to capture a branch had occurred.



Advantages:

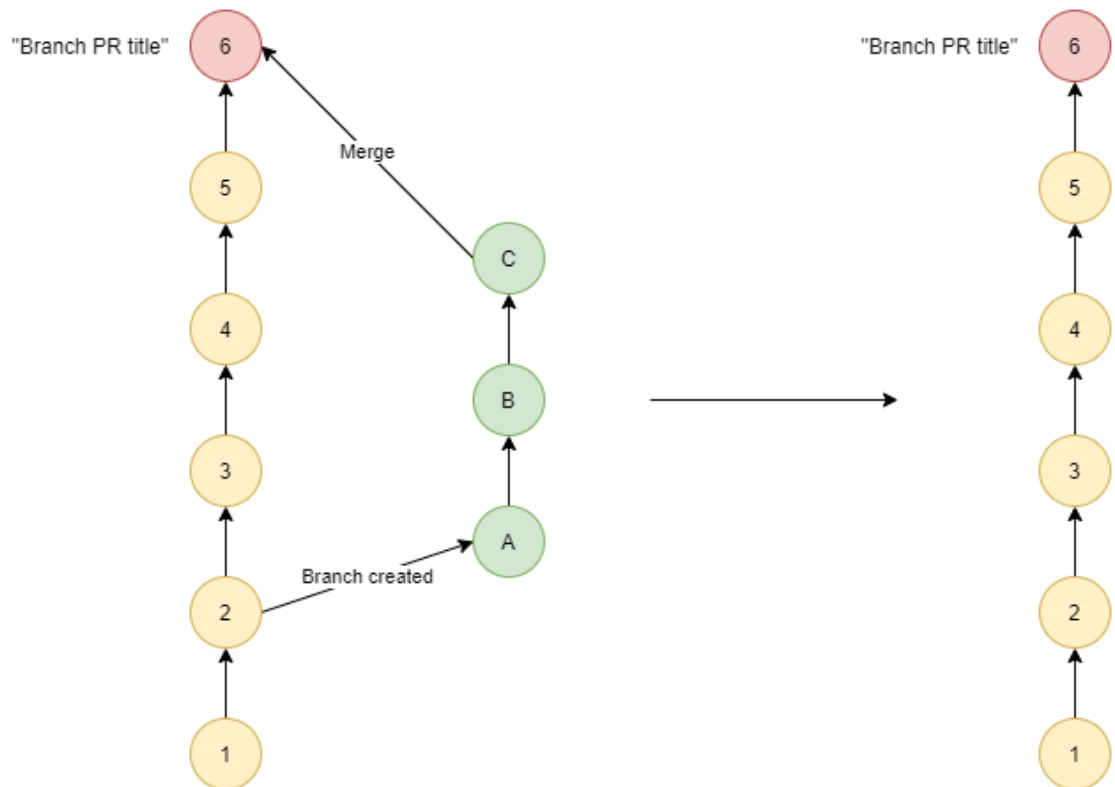
- Keeps a very clean commit history
- Allows us to see each commit that made up the eventual merged changes, no loss of granularity

Disadvantages:

- Can only be done when the base branch hasn't had any new commits, a rarity in a shared repository
- Can be seen as an inaccurate view of history as it hasn't captured that a branch was created, or when it was merged

Squash & Merge

Squash takes all the commits in the branch (A,B,C) and melds them into 1 commit. That commit is then added to the history, but none of the commits that made up the branch are preserved



Advantages:

- Keeps a very clean commit history
- Can look at a single commit to see a full piece of work, rather than shifting through multiple commits in the log

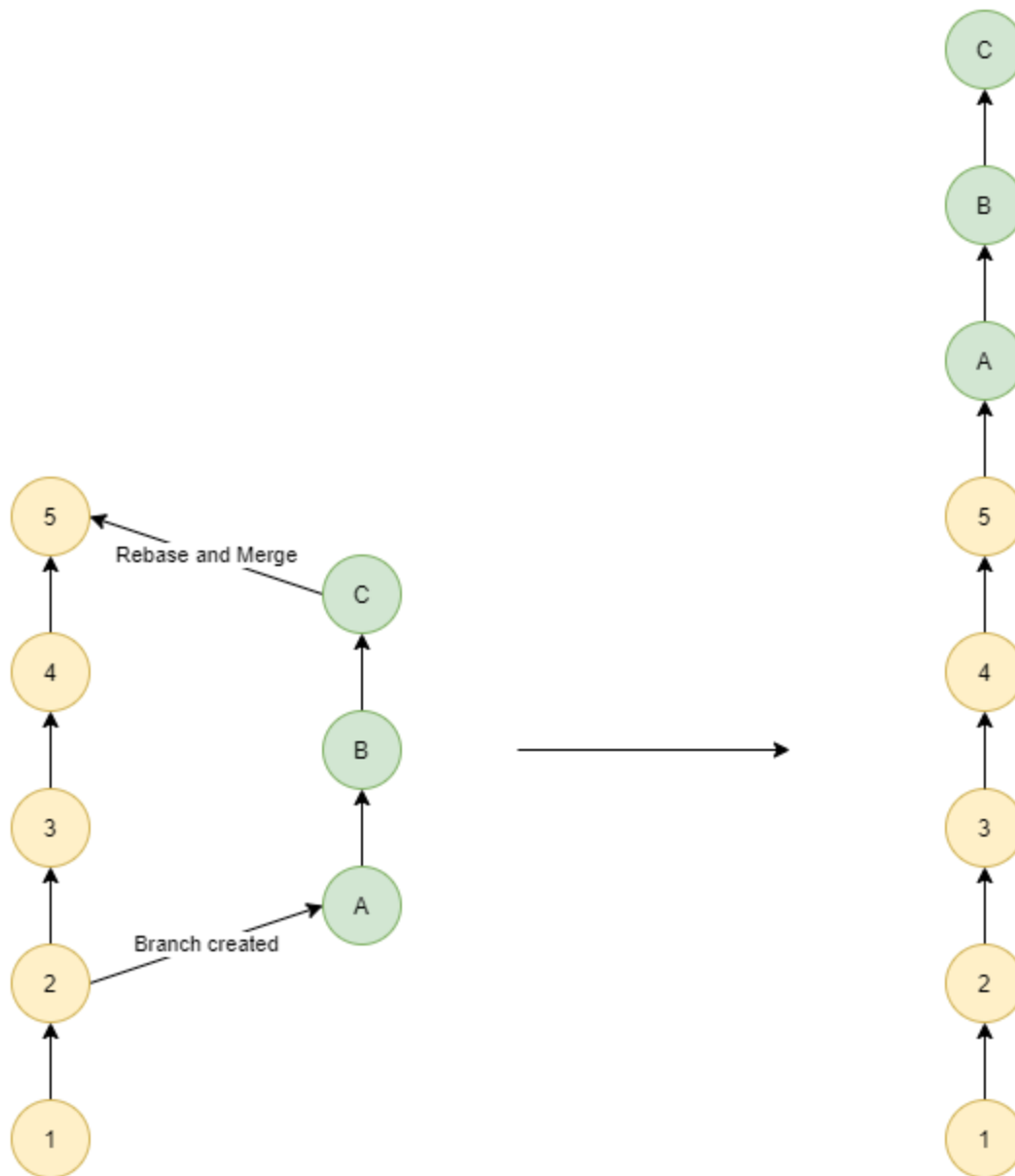
Disadvantages:

- Lost of granularity, any useful detail in those commits that made up the branch is lost, as are any interesting decisions, changes in logic etc captured during the development process.

Rebase & Merge

A rebase and merge will take where the branch was created and move that point to the last commit into the base branch, then reapply the commits on top of those changes.

This is like a fast forward merge, but works when changes have been made into the base branch in the mean while



Advantages:

- Keeps a very clean commit history
- Keeps the individual commit granularity

Disadvantages:

- Can cause frustration as, if someone was to commit to the base branch against before you get to merge, you have to rebase again
- Can be seen as a inaccurate view of history as it hasn't captured that a branch was created, or when it was merged
- Difficult to see which commits relate to which PR / branch

Closing Thoughts

Personally I tend to lean towards Rebase and Merge as it preserves all the history whilst keeping things nice and linear in the logs, however it can be fiddly when working on 1 repository shared by lots of engineers, whilst dealing with rebase conflicts often requires copious amounts of caffeine!

Did I get everything right? Do you have any thoughts on what approach works best? Feel free to ping me on Twitter at [@LukeAMerrett](https://twitter.com/LukeAMerrett).

Update: Q&A

Reverting to a main branch commit after a standard merge

On Twitter Andrew asked:

Nice article, it's good to see this simply explained. One question, with a straightforward merge how would you build from version 4, without the A and B commits
— Andrew Marwood (@mario_wood) [August 2, 2021](#)

To paraphrase, once merged, as the commits are merged in based on time stamp in the history for a branch, how would I later revert to, say, commit "4"

without also getting commits "A" and "B" from the merged branch.

To test how this works I've done a quick check on a dummy repo locally. This is a replicated version of our Sample from the article, each commit was made in the same order in the Sample (1,2,A,3,B,4,C,5)

Before merge using `git log --oneline --graph --all` we can see this structure:

```
PS C:\git\temp\merging-check> git log --oneline --graph --all
* d6fb9d6 (HEAD -> main) 5
* 5549161 4
* 9daa674 3
| * a11a6ff (branch) C
| * cde3697 B
| * 534c9f2 A
|/
* 3edbf33 2
* ce3a0b9 1
```

Following a merge of the `branch` into `main` using `git log --oneline --graph --all` we see:

```
PS C:\git\temp\merging-check> git log --oneline --graph --all
* cda67ba (HEAD -> main) Merge branch 'branch'
| \
| * a11a6ff (branch) C
| * cde3697 B
| * 534c9f2 A
* | d6fb9d6 5
* | 5549161 4
* | 9daa674 3
|/
* 3edbf33 2
* ce3a0b9 1
```

Whilst viewed as a timeline when on the `main` branch we see:


```

cda67ba (HEAD -> main) Merge branch 'branch'
d6fb9d6 5
a11a6ff (branch) C
5549161 4
cde3697 B
9daa674 3
534c9f2 A
3edbf33 2
ce3a0b9 1

```

Back to the question, let's say I wanted to go back to commit "4" excluding the commits "A" and "B" from the merged branch. How would we do that?

Firstly, what happens if I checkout commit "4" using `git checkout 5549161`? The history I see is:

```

5549161 (HEAD) 4
9daa674 3
3edbf33 2
ce3a0b9 1

```

Huh, where have our commits "A" and "B" gone? Looking at the full history (our current location is `head`):

```

* cda67ba (main) Merge branch 'branch'
|
| * a11a6ff (branch) C
| * cde3697 B
| * 534c9f2 A
| * | d6fb9d6 5
| * | 5549161 (HEAD) 4
| * | 9daa674 3
|/
* 3edbf33 2
* ce3a0b9 1

```

By navigating back **before** the merge commit, we won't get any of the individual merged commits in our history. It's only after that merge commit

that they are included in the timeline.

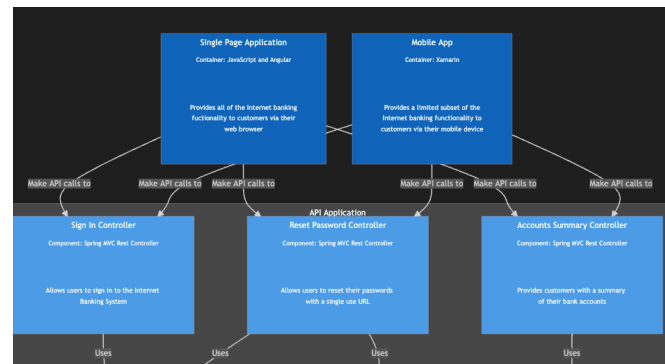
Sign up for more like this.



PyCon UK 2023 Raw Notes

Notes on this year's PyCon UK conference held in Cardiff: talk highlights, quotes &...

29 Sep 2023 8 min read



Building C4 Diagrams in Mermaid

Using MermaidJS to render C4 Diagrams in Markdown

19 Sep 2022 7 min read

Luke Merrett © 2026

Powered by Ghost